Minimized Financial Loss Case Study Report

Arman Azhand

Introduction

For this assignment, the objective is to build a model that predicts for a binary classification task while also minimizing financial losses. Financial losses are incurred via misclassifications – with different losses associated with a false positive than a false negative. Using the anonymized dataset, logistic regression, random forest classifier, and extreme gradient boosting (XGBoost) were explored for this problem. The client did not provide additional information regarding the data, only that the goal is to minimize the financial loss incurred by false positive and false negative predictions. Furthermore, they did not care about model explainability, only financial performance. The dataset had some missing features; thus, imputation used for those missing values before proper preprocessing such as scaling and one-hot encoding for appropriate features was performed before model exploration and tuning.

Data Preparation and Modeling

The anonymized dataset was manually downloaded from an SMU cloud storage management system for this assignment. The data is stored in a comma-separated values (CSV) file. The CSV was loaded in as a dataframe of 160000 rows and 51 columns. Some numeric values were imported as strings, and thus were appropriately converted into float values.

Each feature – except the column representing the binary class – had between 20 to 50 missing values. Missing values were replaced by imputing for the mode of each feature. Of the 50 features, 47 of them contained numeric values and were scaled with sci-kit learn's standard scaler using StandardScaler().

The remaining 3 categorical features were one-hot encoded using pandas get_dummies(). While there were what seem to be spelling mistakes for some of the values in the categorical features, since model explainability is not an issue and the data was anonymized, the spelling mistakes were not addressed.

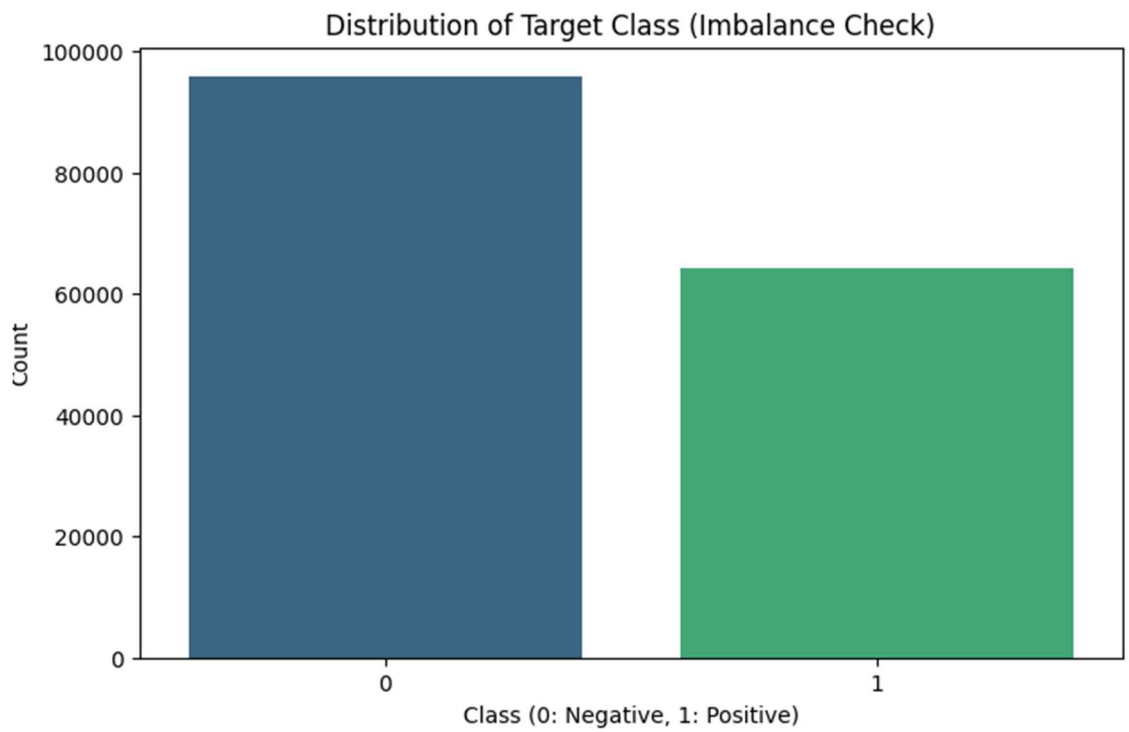

Figure 1: Distribution of Target Class "y" (Imbalance Check). Class 0 has more observations than Class 1 in our dataset of 160000 observations.

Looking at the binary class we will be predicting for, there was some imbalance in our dataset. Class 0 has more observations than Class 1. With proper cross-validation, bias can be mitigated when training each model.

$$Total\ Cost = (False_{Positive} * \$102.33) + (False_{Negative} * \$31.56)$$

Figure 2: Financial cost function. The model that has the smallest cost will be the most beneficial for the problem.

Before model exploration, due to the nature of the problem for minimizing financial losses on false classifications, a scoring metric was created using sci-kit learn's make_scorer(). The cost of a false positive is $102.33 and the cost of a false negative is $31.56. Therefore, the total cost of a model can be calculated by adding the product of the number of false positives and its cost to the product of the number of false negatives and its cost, and the better model would be one that minimizes this cost – or in other words, the model that has the smallest cost.

Three models were explored: XGBoost, Logistic regression, and Random Forest. A grid search 5-fold cross-validation was explored for each model with different parameter values to tune for. Cross-validation was used so that all of our data could be used for training and testing, without testing on data that was used for that training fold. The grid search looked for the model with the lowest financial cost. The results of each model and its hyperparameters were saved and mass exported as a CSV.

Model	Parameters	mean_test_score
XGBoost	{'colsample_bylevel': 0.5, 'learning_rate': 0.05, 'max_depth': 4, 'n_estimators': 500}	-$163,161.52
XGBoost	{'colsample_bylevel': 0.5, 'learning_rate': 0.05, 'max_depth': 6, 'n_estimators': 500}	-$123,508.99
XGBoost	{'colsample_bylevel': 0.5, 'learning_rate': 0.05, 'max_depth': 8, 'n_estimators': 500}	-$105,394.81
XGBoost	{'colsample_bylevel': 0.5, 'learning_rate': 0.1, 'max_depth': 4, 'n_estimators': 500}	-$143,462.82
XGBoost	{'colsample_bylevel': 0.5, 'learning_rate': 0.1, 'max_depth': 6, 'n_estimators': 500}	-$111,942.04
XGBoost	{'colsample_bylevel': 0.5, 'learning_rate': 0.1, 'max_depth': 8, 'n_estimators': 500}	-$98,945.90
XGBoost	{'colsample_bylevel': 0.7, 'learning_rate': 0.05, 'max_depth': 4, 'n_estimators': 500}	-$160,639.03
XGBoost	{'colsample_bylevel': 0.7, 'learning_rate': 0.05, 'max_depth': 6, 'n_estimators': 500}	-$121,452.44
XGBoost	{'colsample_bylevel': 0.7, 'learning_rate': 0.05, 'max_depth': 8, 'n_estimators': 500}	-$105,545.72
XGBoost	{'colsample_bylevel': 0.7, 'learning_rate': 0.1, 'max_depth': 4, 'n_estimators': 500}	-$139,616.54
XGBoost	{'colsample_bylevel': 0.7, 'learning_rate': 0.1, 'max_depth': 6, 'n_estimators': 500}	-$111,437.46
XGBoost	{'colsample_bylevel': 0.7, 'learning_rate': 0.1, 'max_depth': 8, 'n_estimators': 500}	-$98,756.73
XGBoost	{'colsample_bylevel': 0.9, 'learning_rate': 0.05, 'max_depth': 4, 'n_estimators': 500}	-$160,327.06
XGBoost	{'colsample_bylevel': 0.9, 'learning_rate': 0.05, 'max_depth': 6, 'n_estimators': 500}	-$120,914.40
XGBoost	{'colsample_bylevel': 0.9, 'learning_rate': 0.05, 'max_depth': 8, 'n_estimators': 500}	-$103,647.17
XGBoost	{'colsample_bylevel': 0.9, 'learning_rate': 0.1, 'max_depth': 4, 'n_estimators': 500}	-$138,624.61
XGBoost	{'colsample_bylevel': 0.9, 'learning_rate': 0.1, 'max_depth': 6, 'n_estimators': 500}	-$111,482.02
XGBoost	{'colsample_bylevel': 0.9, 'learning_rate': 0.1, 'max_depth': 8, 'n_estimators': 500}	-$95,872.55
LogisticRegression	{'C': 0.1, 'penalty': 'l2'}	-$428,706.33
LogisticRegression	{'C': 1.0, 'penalty': 'l2'}	-$428,871.40
LogisticRegression	{'C': 10.0, 'penalty': 'l2'}	-$428,863.55
RandomForest	{'max_depth': 10, 'max_features': 0.5, 'min_samples_split': 10, 'n_estimators': 300}	-$145,233.73
RandomForest	{'max_depth': 10, 'max_features': 0.5, 'min_samples_split': 6, 'n_estimators': 300}	-$145,572.67
RandomForest	{'max_depth': 10, 'max_features': 0.5, 'min_samples_split': 2, 'n_estimators': 300}	-$145,226.47
RandomForest	{'max_depth': 10, 'max_features': 'sqrt', 'min_samples_split': 10, 'n_estimators': 300}	-$160,379.90
RandomForest	{'max_depth': 10, 'max_features': 'sqrt', 'min_samples_split': 6, 'n_estimators': 300}	-$160,478.41
RandomForest	{'max_depth': 10, 'max_features': 'sqrt', 'min_samples_split': 2, 'n_estimators': 300}	-$160,740.45
RandomForest	{'max_depth': 15, 'max_features': 0.5, 'min_samples_split': 10, 'n_estimators': 300}	-$111,024.38
RandomForest	{'max_depth': 15, 'max_features': 0.5, 'min_samples_split': 6, 'n_estimators': 300}	-$110,538.93
RandomForest	{'max_depth': 15, 'max_features': 0.5, 'min_samples_split': 2, 'n_estimators': 300}	-$110,818.95
RandomForest	{'max_depth': 15, 'max_features': 'sqrt', 'min_samples_split': 10, 'n_estimators': 300}	-$126,547.08
RandomForest	{'max_depth': 15, 'max_features': 'sqrt', 'min_samples_split': 6, 'n_estimators': 300}	-$125,915.50
RandomForest	{'max_depth': 15, 'max_features': 'sqrt', 'min_samples_split': 2, 'n_estimators': 300}	-$125,433.88
RandomForest	{'max_depth': 20, 'max_features': 0.5, 'min_samples_split': 10, 'n_estimators': 300}	-$105,392.38
RandomForest	{'max_depth': 20, 'max_features': 0.5, 'min_samples_split': 6, 'n_estimators': 300}	-$105,190.78
RandomForest	{'max_depth': 20, 'max_features': 0.5, 'min_samples_split': 2, 'n_estimators': 300}	-$105,351.06
RandomForest	{'max_depth': 20, 'max_features': 'sqrt', 'min_samples_split': 10, 'n_estimators': 300}	-$116,242.30
RandomForest	{'max_depth': 20, 'max_features': 'sqrt', 'min_samples_split': 6, 'n_estimators': 300}	-$116,115.86
RandomForest	{'max_depth': 20, 'max_features': 'sqrt', 'min_samples_split': 2, 'n_estimators': 300}	-$114,633.32

Figure 3: Hyperparameter tuning results (parameters and mean financial cost from 5 splits). Note that the best performing parameters for each model have been highlighted.

Results

Looking at the results of each trained model from our grid search cross validation, XGBoost with 500 estimators, 0.9 subsample ratio of columns for each level, learning rate of 0.1, and maximum depth of 8 performed the best with a mean loss of $95,872.55 across all 5 splits. Random Forest performed fairly well, although not as well as the best performing XGBoost. Logistic regression performed fairly poorly, averaging almost 4x the financial loss compared to the best performing XGBoost and Random Forest classifiers.

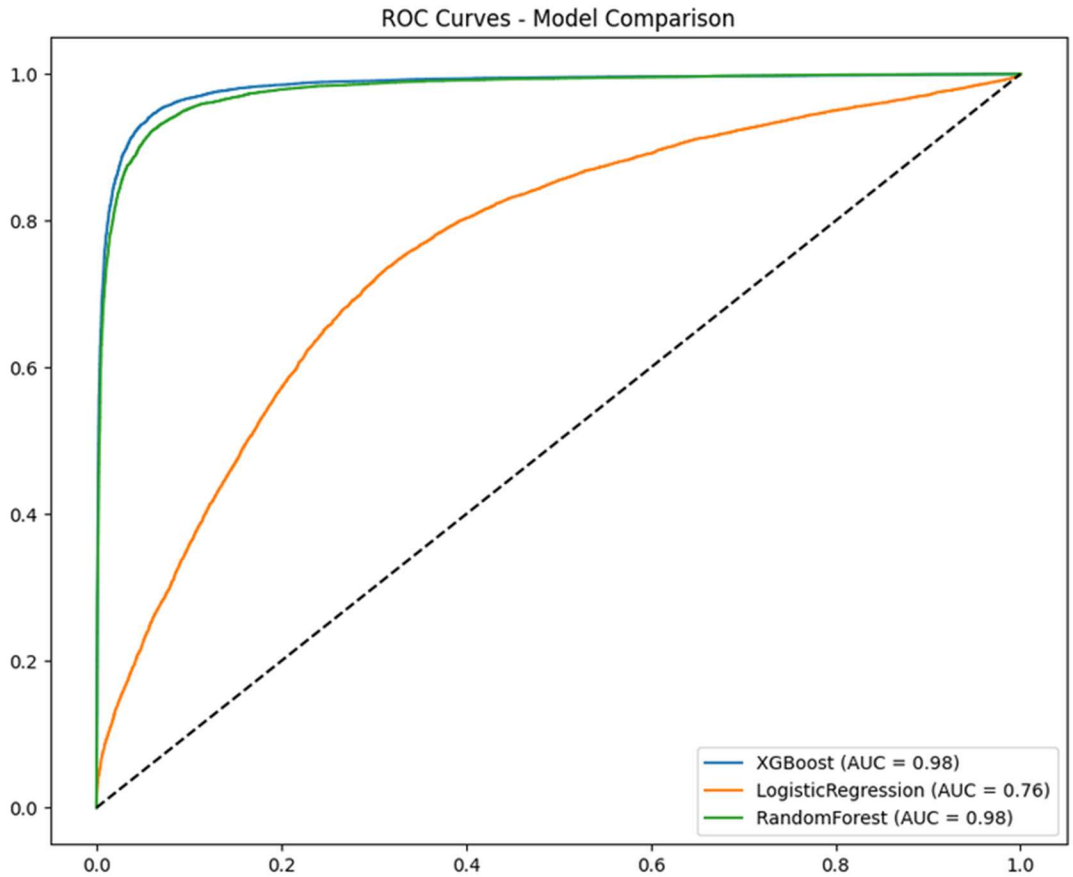

Figure 4: ROC Curves (best performing model for each model type). Note that Logistic Regression performed very poorly, which matches the financial loss it returns, whereas XGBoost and Random Forest have very high areas under the curve, which also matches what we would expect looking at the financial loss they return respectively. XGBoost shows a slightly higher AUC as dictated by the gap we see between the blue (XGBoost) and green (RandomForest) lines.

Due to the boasting the smallest mean financial loss and the highest area under the curve for the receiver operating characteristic (ROC) curves, the aforementioned XGBoost model was chosen as the final model. A final 5-fold cross validation was performed on the tuned XGBoost and the reported financial loss on all 160000 observations was $634,186.83 – where 4091 false positive cost $418,632.03 and 6830 false negatives cost $215,554.90.

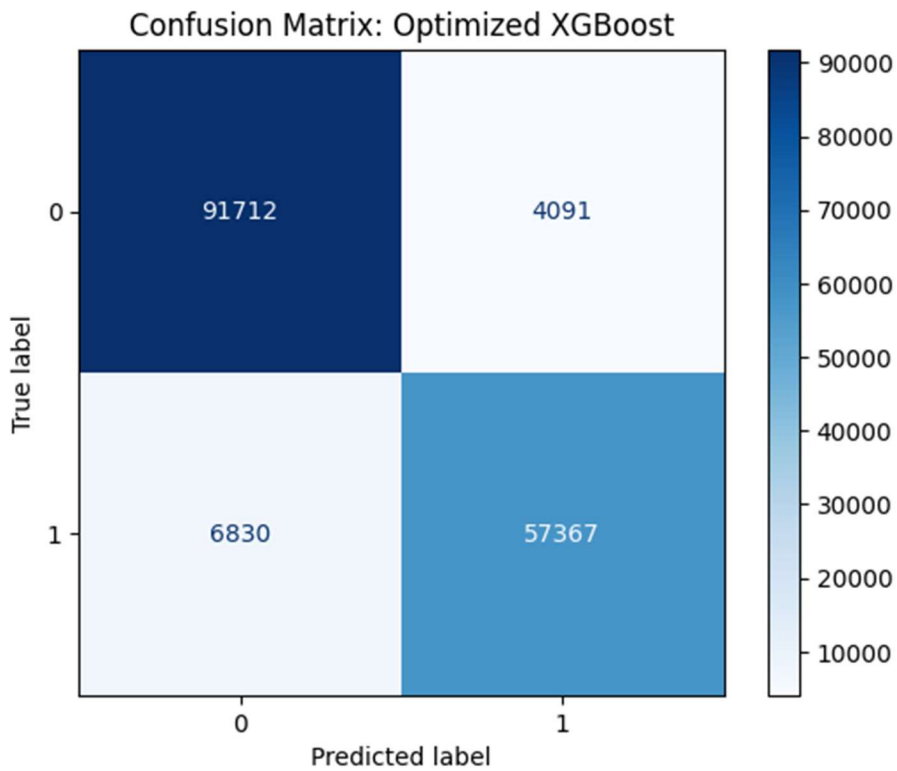

Figure 5: Confusion matrix (for the most optimized XGBoost classifier). Note that due to the class imbalance in our data, there are less Class 1 to predict for than Class 0.

Classification Report				
	Precision	Recall	F1-score	Support
0	0.93	0.96	0.94	95803
1	0.93	0.89	0.91	64197
Accuracy			0.93	160000
Macro Average	0.93	0.93	0.93	160000
Weighted Average	0.93	0.93	0.93	160000

Figure 6: Classification report (for the most optimized XGBoost classifier). As we expect by looking at the confusion matrix for the same model, the f1-score for both classes is very close to one another, with Class 0's being higher due to the recall.

Summary

The best classifier from our grid search cross validation was the tuned XGBoost – which 500 estimators, 0.9 subsample ratio of columns for each level, learning rate of 0.1, and maximum depth of 8, returning an average validation loss of -$95,872.55. After cross-validating for our chosen model, the reported financial loss on all 160000 observations was $634,186.83 – where 4091 false positive cost $418,632.03 and 6830 false negatives cost $215,554.90.

Since model explainability is not a concern, and minimizing financial cost due to misclassification is most important, the tuned XGBoost model is the most appropriate model for this classification task.

Before building our models, data importation of the csv file, converting improperly imported data, scaling for numeric data, and one-hot encoding categorical data was performed. 5-fold cross validation was used such that all 160000 observations could be used for training, where 20% of the data was held off for testing and the rest was used for training, each split.